

Практическое занятие №2

Архитектура нейронных сетей

Нейронные сети. Искусственный нейрон. Активационные функции. Базовые архитектуры нейронных сетей.

В последние годы нейронные сети стали одним из наиболее популярных методов для решения различных задач, таких как классификация изображений, прогнозирование временных рядов, обработка естественного языка, генерация контента и т.д. Они «умеют» извлекать признаки из данных и на основе этих признаков принимать решения, что делает их особенно полезными в сфере искусственного интеллекта.

Python является одним из самых популярных языков программирования для создания нейронных сетей, благодаря своей простоте и богатой экосистеме библиотек машинного обучения.

Архитектура нейронных сетей

Архитектура нейронных сетей описывает структуру нейронной сети и определяет, как она будет обрабатывать входные данные и выдавать выходные значения. Существует несколько типов архитектур нейронных сетей, каждый из которых предназначен для решения определенных задач.

Перцептрон

Перцептрон — это один из самых простых типов нейронных сетей, который состоит из одного или нескольких слоев нейронов. Каждый нейрон в перцептроне имеет свои веса и смещение, которые позволяют ему обрабатывать входные данные и выдавать выходные значения.

Перцептроны часто используются для задач классификации, таких как определение, является ли изображение котом или собакой.

Сверточные нейронные сети

Сверточные нейронные сети особенно хорошо подходят для обработки изображений. Они имеют несколько слоев, включая сверточные слои, пулинг слои и полносвязные слои.

Сверточные слои используются для извлечения признаков из изображений, пулинг слои уменьшают размерность выходных данных, а полносвязные слои используются для принятия окончательного решения на основе извлеченных признаков.

Рекуррентные нейронные сети

Рекуррентные нейронные сети – это тип нейронных сетей, который используется для работы с последовательными данными, такими как звуковые сигналы или текстовые данные. Рекуррентные слои в этих нейронных сетях позволяют нейронной сети запоминать информацию из предыдущих шагов и использовать ее для принятия решения на текущем шаге. Это позволяет рекуррентным нейронным сетям работать с данными разной длины и предсказывать последующие значения в последовательности.

Определение весов и смещений

Когда нейронная сеть получает на вход некоторые данные, она проходит через несколько слоев, состоящих из нейронов. Каждый нейрон обрабатывает данные и выдает некоторый результат, который передается следующему слою нейронов. Чтобы нейронная сеть могла правильно работать, ей необходимо научиться извлекать признаки из данных, то есть определять, какие входные значения наиболее важны для принятия решения.

Для этого каждый нейрон в нейронной сети имеет свой вес и смещение. Веса определяют, насколько каждый входной параметр важен для определения выхода нейрона, а смещение позволяет нейрону изменять свой выход в зависимости от входных данных.

В процессе обучения нейронная сеть корректирует значения весов и смещений таким образом, чтобы минимизировать ошибку на выходе. Для этого используются различные методы оптимизации, такие как стохастический градиентный спуск, а также различные функции потерь, которые позволяют измерить ошибку на выходе нейронной сети.

Функция активации

Функция активации играет ключевую роль в работе нейронной сети. Она применяется к выходу каждого нейрона и определяет, должен ли он быть активирован и передать свое значение на следующий слой нейронов.

Существует несколько типов функций активации, но одной из самых популярных является функция **ReLU** (Rectified Linear Unit). Она имеет вид $f(x) = \max(0, x)$ и позволяет нейрону передавать значение, если оно положительно, а иначе – передавать нулевое значение.

Другие функции активации, такие как сигмоида, также используются в нейронных сетях, но они менее эффективны, чем функция **ReLU**, особенно при работе с глубокими нейронными сетями.

При создании своей нейросети на Python необходимо выбрать подходящую функцию активации в зависимости от задачи, которую вы хотите решить. Кроме того, важно убедиться, что функция активации выбрана правильно, чтобы избежать проблем, таких как затухание градиента.

Функции потерь и оптимизация

После выбора функции активации необходимо выбрать функцию потерь, которая будет измерять ошибку нейронной сети в процессе обучения. Функция потерь должна быть выбрана в зависимости от задачи, которую вы хотите решить. Например, для задачи классификации могут быть использованы функции потерь, такие как кросс-энтропия или среднеквадратичная ошибка.

Кроме того, необходимо выбрать метод оптимизации для обучения нейронной сети. Оптимизатор используется для изменения весов нейронной сети в процессе обучения, чтобы минимизировать функцию потерь. Один из наиболее популярных оптимизаторов — это алгоритм стохастического градиентного спуска (SGD). Он обновляет веса нейронной сети в направлении, противоположном градиенту функции потерь.

Существуют и другие методы оптимизации, такие как Adam и Adagrad, которые могут быть более эффективны в некоторых случаях. При выборе оптимизатора также следует учитывать задачу и характеристики данных.

Выбор правильной функции потерь и оптимизатора — это важный шаг при создании нейронной сети. Они должны быть выбраны в соответствии с задачей и характеристиками данных, чтобы обеспечить наилучшую производительность нейронной сети.

Создание простой нейросети на Python

Рассмотрим создание простой нейросети на Python для решения определенной задачи. Возьмем таблицу с 4 столбцами: 3 из них будут входами, а последний — выходом.

Первый вход	Второй вход	Третий вход	Результат
0	0	1	0
1	1	1	1
1	0	1	1
0	1	1	0

В этом случае результат всегда будет равен значению первого входа. Попробуем создать и обучить нейросеть, которая будет на основе трёх входов выдавать ожидаемый результат. У нас получится нейросеть перцептрон со следующей архитектурой:

На вход нейросеть получает 3 параметра. Во время обучения нейросеть выберет оптимальные веса. В конце нейрон прогонит перемноженные веса и параметры через функцию активации.

numpy

Numpy —

библиотека для математических операций. В неё вложены в том числе и матричные операции, которые особенно важны в нейронных сетях.

импортируйте библиотеку в код:

```
import numpy as np
```

Напишем функцию активации — сигмоиду:

```
def sigmoid(x):  
    return 1/(1 + np.exp(-x))
```

Существуют различные функции, и, как мы уже упоминали, их выбор важен при проектировании больших нейросетей. В нашем случае выбор не настолько принципиален.

Теперь создадим массив с обучающими данными:

```
training_inputs = np.array ([[0,0,1],  
[1,1,1],  
[1,0,1],  
[0,1,1]])  
  
training_outputs = np.array([[0,1,1,0]]).T
```



В переменной `training_inputs` хранятся входные данные, а в `training_outputs` — выходы.

Выберем случайные веса:

```
np.random.seed(1)
synaptic_weights = 2 * np.random.random((3,1)) - 1
```

И обучим нейросеть:

```
for i in range(10000):
    input_layer = training_inputs
    outputs = sigmoid(np.dot(input_layer, synaptic_weights))

    err = training_outputs - outputs
    adjustments = np.dot(input_layer.T, err * (outputs))
    synaptic_weights += adjustments

print("веса после обучения")
print(synaptic_weights)
```

Вывод:

```
веса после обучения
[[14.09344278]
 [-0.18776581]
 [-4.50486337]]
```



Что происходит в этом отрезке кода: нейросеть итерационно подбирает оптимальные веса. С каждой итерацией она становится всё ближе к правильным значениями.

Проверим текущий результат:

```
print("result")
print(outputs)
```



Вывод:

```
result
[[0.01093477]
 [0.99991734]
 [0.99993149]
 [0.00907983]]
```

И, чтобы убедиться в работоспособности нейросети, проверим её на незнакомом примере:

```
new_input = np.array([[0,0,1]])

print(sigmoid(np.dot(new_input, synaptic_weights)))
```

Вывод:

```
[[0.01093422]]
```

The screenshot shows the myCompiler Python IDE interface. The top bar includes the myCompiler logo, a language dropdown set to English, and links for Recent, Login, and Sign up. Below the top bar is a title input field. The main editor area shows a Python script for training a simple neural network. The script imports numpy, defines a sigmoid function, sets training inputs and outputs, initializes synaptic weights, and performs 10,000 iterations of training. After training, it prints the weights, the training outputs, and the result for a new input. The right sidebar shows the 'Program input' field and the 'Output' panel. The output panel displays the results of the training and the test run.

```
1 import numpy as np
2
3 def sigmoid(x):
4     return 1/(1 + np.exp(-x))
5
6 training_inputs = np.array ([[0,0,1],
7 [1,1,1],
8 [1,0,1],
9 [0,1,1]])
10
11 training_outputs = np.array([[0,1,1,0]]).T
12
13 np.random.seed(1)
14 synaptic_weights = 2 * np.random.random((3,1)) - 1
15
16 for i in range(10000):
17     input_layer = training_inputs
18     outputs = sigmoid(np.dot(input_layer, synaptic_weights))
19
20     err = training_outputs - outputs
21     adjustments = np.dot(input_layer.T, err * (outputs))
22     synaptic_weights += adjustments
23
24 print("веса после обучения")
25 print(synaptic_weights)
26
27 print("result")
28 print(outputs)
29
30 print("Проверка работоспособности нейросети на незнакомом примере")
31 new_input = np.array([[0,0,1]])
32 print(sigmoid(np.dot(new_input, synaptic_weights)))
```

Output

```
веса после обучения
[[14.09344278]
 [-0.18776581]
 [-4.50486337]]
result
[[0.01093477]
 [0.99991734]
 [0.99993149]
 [0.00907983]]
Проверка работоспособности нейросети на незнакомом примере
[[0.01093422]]

[Execution complete with exit code 0]
```

Рис. 1. Пример простейшей нейросети на Python

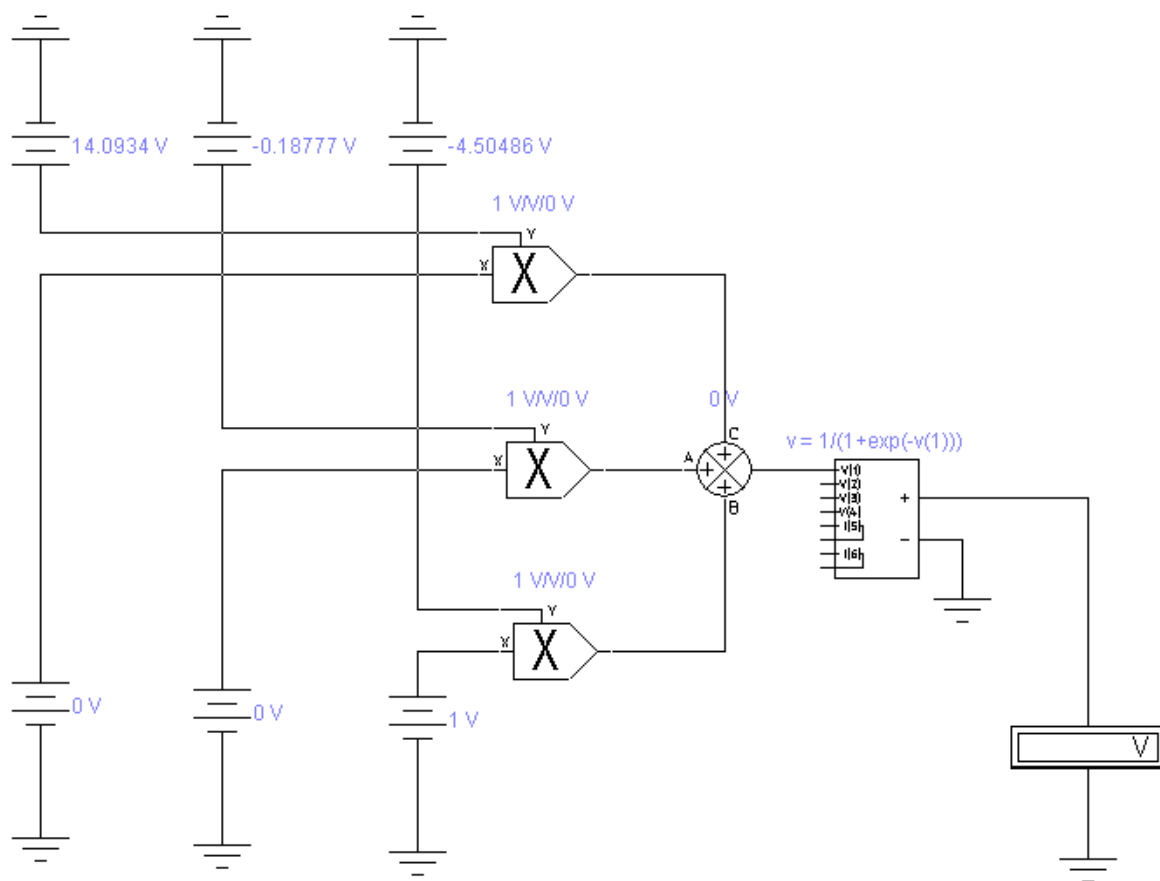


Рис. 2. Аппаратная реализация обученного перцептрона в Electronics Workbench.

Содержание работы.

Ответьте на вопросы и выполните следующие задания:

1. Что называется нейронной сетью?
2. Что называют искусственным нейроном?
3. Какие модели искусственных нейронов существуют?
4. Какие существуют типы функций активации?
5. Дайте определение “перцептрон”.
6. Перечислите базовые архитектуры нейронных сетей.
7. Привести код на Python простейшей нейросети.
8. Самостоятельно провести эксперименты с простейшей нейросетью на иных произвольных входных данных. Получите веса после обучения.
9. Осуществите аппаратную реализацию обученного перцептрона в Electronics Workbench. Проанализируйте полученный результат.